**Gebze Technical University**

CSE 341 — Concepts of Programming Languages

Spring 2026

# Project Part 2 — Design Specification (D1)

# ScoreScript

*A Domain-Specific Language for Student Grade Management*

Student Number: **230104004090**

Name: **FATİH EMRE OĞAN**

Submission Date: **22 May 2026**

## 4.1 Language Overview

ScoreScript came to mind while I was working on this assignment. I thought: what if there was a language made for everyone involved in academic grading — not just the instructor, but also teaching assistants who track quiz scores, and academic advisors who follow a student's progress across courses?

ScoreScript is a domain-specific language for academic grade management. With ScoreScript, you can define students with their id, name and grades, group them into classes, apply grade curves, compute averages, rank students and print transcripts. The syntax is simple and readable — even someone with little programming experience can use it.

When designing ScoreScript, I prioritized readability and writability.

**Sample Program:**

```
student s1 {
    id: "230104004090",
    firstName: "Fatih Emre",
    lastName: "OGAN",
    grades: { math: 85, science: 90, history: 72 }
}
group ClassZ23 = [s1, s2]
curve ClassZ23 by 5
average ClassZ23
rank ClassZ23
transcript s1
```

**Language Evaluation:**

ScoreScript prioritizes readability, writability, and reliability. Grade management is a repetitive and detail-sensitive task, so the language is designed to be concise and easy to use without requiring deep programming knowledge. Reliability is critical: a grading error directly affects a student's academic record, which is why ScoreScript enforces strict field ordering and rejects malformed programs with precise error messages. Performance and cost are knowingly sacrificed — a typical class has at most a few hundred students, so execution speed is simply not a concern for this domain.

## 4.2 Lexical Structure

The ScoreScript lexer recognises the following token categories. Each category is defined by a regular expression or explicit enumeration. The lexer processes tokens in the order listed; multi-character operators are matched greedily before single-character ones, and FLOAT literals are attempted before INT to prevent a value such as 60.0 from being tokenised as INT(60) followed by a dot and INT(0). Single-line comments beginning with // are stripped by the lexer and produce no tokens.

### Keywords

Keywords are reserved identifiers. They are matched before the IDENT rule and cannot be used as variable or function names.

| Category | Tokens |
|---|---|
| Statement / declaration | student group func for in if else return print by |
| Operations | curve average rank transcript |
| Type keywords | int float string bool void |
| Student field keywords | id firstName lastName grades |
| Boolean literals | true false |

### Identifiers

User-defined names follow the rule: IDENT = [a-zA-Z][a-zA-Z0-9_]*

An IDENT that matches any keyword is classified as that keyword, not as IDENT.

### Literals

| Token | Pattern | Examples |
|---|---|---|
| INT | [0-9]+ | 0 85 100 |
| FLOAT | [0-9]+ "." [0-9]+ | 60.0 3.14 99.5 |
| STRING | "\"" { char } "\"" | "Fatih Emre" "OGAN" |

The FLOAT rule is attempted before INT. An unterminated STRING is a lexical error reported with the line number.

### Operators

| Category | Tokens |
|---|---|
| Arithmetic | + - * / |
| Unary | - ! |
| Relational | < <= > >= |
| Equality | == != |
| Logical | && \|\| |
| Assignment / arrow | = -> |

### Separators

| Token | Character |
|---|---|
| LBRACE / RBRACE | { } |
| LPAREN / RPAREN | ( ) |
| LBRACKET / RBRACKET | [ ] |
| COMMA | , |
| COLON | : |

Comments: COMMENT = "//" { any character except newline }

ScoreScript supports single-line comments only. Block comments are not supported. Comments are consumed by the lexer and produce no tokens.

## 4.3 Syntax

The complete grammar of ScoreScript is given below in EBNF following the conventions of Sebesta §3.1–3.2. Terminal symbols appear in double quotes or as token names in ALL CAPS. The metasymbols { } denote zero or more repetitions; [ ] denote an optional element; | separates alternatives; parentheses group.

The grammar is unambiguous by construction. Every production is uniquely identified by its leading token, so the parser requires at most one token of lookahead (LL(1) property).

### Program Structure

```
<program>      ::= <decl_section> <stmt_section>
<decl_section> ::= { <student_decl> | <group_decl> | <var_decl> | <func_decl> }
<stmt_section> ::= { <stmt> }
```

All declarations must precede all statements. A program that places a statement before the end of its declarations is rejected at parse time.

### Declarations

```
<student_decl> ::= "student" IDENT "{"
                   "id" ":" STRING ","
                   "firstName" ":" STRING ","
                   "lastName" ":" STRING ","
                   "grades" ":" "{" <grade_list> "}" "}"
<grade_list>   ::= <grade_entry> { "," <grade_entry> }
<grade_entry>  ::= IDENT ":" INT
<group_decl>   ::= "group" IDENT "=" "[" <ident_list> "]"
<ident_list>   ::= IDENT { "," IDENT }
<var_decl>     ::= <value_type> IDENT "=" <expr>
<func_decl>    ::= "func" IDENT "(" [ <param_list> ] ")" "->" <type> "{"
<func_body> "}"
<param_list>   ::= <param> { "," <param> }
<param>        ::= IDENT ":" <value_type>
<func_body>    ::= { <stmt> }
```

### Statements

```
<stmt>       ::= <operation> | <if_stmt> | <for_stmt> | <print_stmt>
               | <return_stmt> | <var_decl> | <assign_stmt> | <call_stmt>
<operation>  ::= "curve" IDENT "by" <expr>
               | "average" IDENT
               | "rank" IDENT
               | "transcript" IDENT
<if_stmt>    ::= "if" <expr> "{" { <stmt> } "}"
               { "else if" <expr> "{" { <stmt> } "}" }
               [ "else" "{" { <stmt> } "}" ]
<for_stmt>   ::= "for" "student" IDENT "in" IDENT "{" { <stmt> } "}"
<print_stmt> ::= "print" <expr>
<return_stmt>::= "return" <expr>
<assign_stmt>::= IDENT "=" <expr>
<call_stmt>  ::= IDENT "(" [ <arg_list> ] ")"
```

### Types

```
<type>       ::= <value_type> | "void"
<value_type> ::= "int" | "float" | "string" | "bool"
```

The void type is separated from value types at the grammar level. Parameters use <value_type>, not <type>, so a parameter declared void is a parse error.

### Expressions

```
<or_expr>         ::= <and_expr> { "||" <and_expr> }
<and_expr>        ::= <equality_expr> { "&&" <equality_expr> }
<equality_expr>   ::= <relational_expr> { ("==" | "!=") <relational_expr> }
<relational_expr> ::= <additive_expr> { ("<" | "<=" | ">" | ">=") <additive_expr> }
<additive_expr>   ::= <mult_expr> { ("+" | "-") <mult_expr> }
<mult_expr>       ::= <unary_expr> { ("*" | "/") <unary_expr> }
<unary_expr>      ::= "-" <unary_expr> | "!" <unary_expr> | <primary>
<primary>         ::= INT | FLOAT | STRING | "true" | "false"
                    | IDENT | <func_call_expr> | "(" <expr> ")"
<func_call_expr>  ::= IDENT "(" [ <arg_list> ] ")"
<arg_list>        ::= <expr> { "," <expr> }
```

### Operator precedence table:

| Level | Grammar Rule | Operator(s) | Associativity |
|---|---|---|---|
| 1 (tightest) | <unary_expr> | - ! | Right |
| 2 | <mult_expr> | * / | Left |
| 3 | <additive_expr> | + - | Left |
| 4 | <relational_expr> | < <= > >= | Left |
| 5 | <equality_expr> | == != | Left |
| 6 | <and_expr> | && | Left |
| 7 (loosest) | <or_expr> | \|\| | Left |

## 4.4 Semantics

This section gives an operational semantics (Sebesta §3.5) for two non-trivial constructs of ScoreScript: average and curve. The semantics is given in big-step (natural) operational form.

### State model

A ScoreScript program state is the triple σ = ( σ_V , σ_S , σ_G ) where:

- σ_V is the variable environment, a partial function from variable names to values (int, float, string, bool);
- σ_S is the student environment, a partial function from student name to grade map γ : Subject → Z;
- σ_G is the group environment, a partial function from group name to ordered list of student names.

The auxiliary function: avg(γ) = ( Σ_{k ∈ dom(γ)} γ(k) ) / |dom(γ)| provided |dom(γ)| > 0

### 4.4.1 Semantics of average X

average X computes the mean of per-student averages and prints it. It is a pure observation — it does not modify σ. Let σ_G(X) = [s_1, ..., s_n].

```
   σ_G(X) = [s_1, ..., s_n]      n > 0
   for all i ∈ {1..n}.  A_i = avg( σ_S(s_i) )
-------------------------------------------------------  (E-Avg)
< average X , σ >  =>  σ ,  print( (Σ_{i=1..n} A_i)/n )
```

The state is unchanged in the conclusion, formalising that average has no side effect. The premise n > 0 handles the empty-group case:

```
     σ_G(X) = [ ]
---------------------------------  (E-Avg-Empty)
< average X , σ >  =>  error
```

### 4.4.2 Semantics of curve X by e

curve X by e adds a computed amount to every grade of every student in X, clamping each result to [0, 100]. Unlike average, curve mutates the student environment.

clamp(x) = 0 if x < 0;  100 if x > 100;  x otherwise

The expression e is evaluated once, before any grade is modified. Let σ_G(X) = [s_1, ..., s_k]:

```
< e , σ > => m    m ∈ Z    σ_G(X) = [s_1, ..., s_k]
σ_S' = σ_S with σ_S'(s_i)(κ) = clamp( σ_S(s_i)(κ) + m )
              for each i ∈ {1..k} and κ ∈ dom(σ_S(s_i))
-------------------------------------------------------  (E-Curve)
< curve X by e , σ > => ( σ_V , σ_S' , σ_G )
```

Only σ_S is rewritten; σ_V and σ_G are preserved.

## 4.5 Type System

This section answers the type-system questions of Sebesta Chapter 6 for ScoreScript.

### 4.5.1 Primitive types (Sebesta §6.2–6.4)

| Type | Value set | Example literals |
|---|---|---|
| int | integers (bounded subset of Z) | 0, 5, 85 |
| float | IEEE-754 double-precision reals | 0.0, 59.5, 90.0 |
| string | finite sequences of characters | "OGAN", "AA" |
| bool | { true, false } | true, false |

### 4.5.2 Strong typing (Sebesta §6.12)

ScoreScript is strongly typed. Every variable, parameter and expression has a single statically determined type. The type checker rejects, before execution, every type violation: undeclared names, type mismatches in declarations or assignments, non-bool conditions, arity or type errors in function calls, void results used as values, non-group names passed to domain operations, and non-student names passed to transcript. There is no cast or untyped escape hatch.

### 4.5.3 No implicit coercion (Sebesta §7.4)

ScoreScript performs no implicit type coercion. int and float are not mixed: every arithmetic or relational operator requires both operands to have identical type. An expression such as 1 + 2.0 or avg >= 90 (float compared with int literal) is a type error. The programmer must write both operands with the same type (e.g. 90.0 instead of 90).

### 4.5.4 Type equivalence — name equivalence (Sebesta §6.14)

ScoreScript uses name equivalence: two types are equivalent if and only if their names are identical. The four primitive types are pairwise incompatible. There are no type aliases or anonymous structural types, so name equivalence is the natural and stricter choice.

### 4.5.5 Structured type — the per-student grade map (Sebesta §6.5–6.7)

ScoreScript's structured type is the grade map: a finite mapping from subject names to integer scores (e.g. grades: { math: 85, science: 90 }). It is closest to an associative-array / record hybrid. Key design decisions:

- Field set is fixed at declaration time; curve mutates values but not the key set.
- Element type is uniform (int), so no per-entry type checking is needed.
- No user-level indexing operator exists; the map is consumed only by the four domain operations, removing the subscript-out-of-range error class entirely.
- The grade map is not a first-class declarable type, so name equivalence never has to compare two grade-map types.

## 4.6 Names, Binding, Scope, and Lifetime

**Legal Identifiers:**

In ScoreScript, an identifier must start with a letter and can be followed by letters, digits, or underscores. So s1, ClassZ23, and letter_grade are valid, but 2fast and _temp are not. Reserved keywords like if, student, and curve cannot be used as identifiers.

**Binding Time:**

Compile time: Variable types, function signatures, and student/group names are bound at compile time. Runtime: Variable values are bound at runtime. The for-loop iterator variable is re-bound at each iteration during execution.

**Scoping:**

ScoreScript uses static scoping. This means a variable's scope is determined by where it is written in the code. If we used dynamic scoping, the same function could give different results depending on which function called it. For example, if two functions both have a variable named avg, they could interfere with each other. This would make grade calculations unpredictable, which is exactly what we want to avoid in a grading language.

**Lifetime:**

Top-level declarations (students, groups, global variables) have static lifetime — they are created when the program starts and exist until it ends. Local variables declared inside a function body have stack-dynamic lifetime. The for-loop variable also has stack-dynamic lifetime — it is not visible outside the loop. ScoreScript has no explicit-heap-dynamic variables.

## 4.7 Expressions and Assignment

### 4.7.1 Operator precedence and associativity

The grammar fixes precedence by stratifying expression non-terminals. From tightest binding to loosest:

| Level | Operator(s) | Associativity |
|---|---|---|
| 1 (tightest) | unary -, unary ! | right |
| 2 | * / | left |
| 3 | + - | left |
| 4 | < <= > >= | left |
| 5 | == != | left |
| 6 | && | left |
| 7 (loosest) | \|\| | left |

All binary operators are left-associative. Unary prefix operators are right-associative, permitting !!b or - -x. The precedence table is a direct reading of the grammar's production hierarchy.

### 4.7.2 Short-circuit evaluation of && and || (Sebesta §7.7)

The Boolean operators && and || use short-circuit (lazy) evaluation, left to right:

- For a && b: if a is false, the whole expression is false and b is not evaluated.
- For a || b: if a is true, the whole expression is true and b is not evaluated.

All other binary operators evaluate both operands left-then-right.

### 4.7.3 Assignment is a statement, not an expression (Sebesta §7.8)

In ScoreScript, assignment IDENT = <expr> is a statement, not an expression. Consequences:

- No chained assignment (a = b = c is not valid).
- An assignment cannot be mistaken for a comparison inside a condition (if x = 5 is not well-formed).
- Assignment has no place in operand-evaluation-order reasoning.

This removes the assignment-in-expression defect class entirely, at the cost of some expressive convenience.

## 4.8 Design Rationale

This section gives the rationale for five design decisions in ScoreScript.

#### 1. No implicit coercion — int + float is an error

When I was designing ScoreScript, I wanted to make sure that grade calculations are always predictable. If the language allowed mixing int and float silently, a programmer could accidentally write avg >= 90 instead of avg >= 90.0 and the program would still run  but with a hidden type mismatch underneath. I decided that this kind of silent conversion is more dangerous than useful in a grading context so ScoreScript simply rejects it. Yes, it means you have to write 90.0 instead of 90 in a few places and I had to fix exactly this in program2.ss. But I think that small extra effort is worth it because the language catches the mistake at compile time instead of letting it slip through.

#### 2. void is separated from the value types at the grammar level

I separated void from the value types in the grammar itself, not just in the type checker. This means you cannot even write void x = ... as a declaration  the parser rejects it before the type checker ever runs. My main reason was that accidentally using the result of a void function as a value is a very easy mistake to make and I wanted the language to catch it as early as possible. The trade-off is that the grammar has two separate type categories instead of one which makes a few rules slightly more complex. But I think that is a small price to pay for eliminating a whole class of errors at the structural level.

### 3. Fixed field order in student declarations

I made the field order in student declarations fixed: id, firstName, lastName, grades, always in that order. One reason was practical  it keeps the parser simple because each field is expected at a specific position so there is no need to track which fields have appeared or handle optional orderings. The other reason was readability: when every student block looks exactly the same, it is much easier to scan through a program and spot mistakes. The downside is that if you accidentally write lastName before firstName, it is a hard parse error. But I think that is actually a good thing  it catches typos immediately instead of silently accepting a wrong order.

### 4. Static scoping

I chose static scoping for ScoreScript because I wanted the behavior of every function to be predictable just by reading its source code. With dynamic scoping, a function like letterGrade could behave differently depending on which function called it because it might accidentally pick up a local variable from the caller. In a grading language that is a serious problem  the same function should always give the same result for the same input. Static scoping guarantees that. The trade-off is that a function cannot implicitly see the caller's local variables so if a function needs a value it must either receive it as a parameter or read it from a global. I think that is actually cleaner anyway because it makes dependencies explicit.

### 5. The for loop requires the student keyword

The for loop in ScoreScript requires the student keyword so you always write for student s in G instead of just for s in G. I added this requirement because ScoreScript is a domain-specific language and iteration only makes sense over groups of students. The keyword makes that intent clear right at the point where you read the loop you immediately know that s is a student, not an index or a grade value. It also helps the parser because the student keyword makes the start of a for statement unambiguous. The only cost is a bit of extra verbosity but I think that is acceptable since it makes the code more readable and the domain meaning more explicit.